

HERA: A Bandwidth-efficient Accelerator for Fully Homomorphic Encryption on HBM-enabled FPGA

Zhihan Xu
University of Southern California
Los Angeles, USA
zhihanxu@usc.edu

Rajgopal Kannan
DEVCOM Army Research Office
Los Angeles, USA
rajgopal.kannan.civ@army.mil

Viktor Prasanna
University of Southern California
Los Angeles, USA
prasanna@usc.edu

Abstract

Fully Homomorphic Encryption (FHE) enables privacy-preserving computation on encrypted data. However, it incurs massive computation and DRAM traffic overheads, making hardware acceleration essential. Existing FPGA-based solutions offer limited exploration of bandwidth utilization and memory optimizations, leaving room for further performance improvements.

In this paper, we present HERA, a bandwidth-efficient accelerator for FHE on High Bandwidth Memory (HBM)-enabled FPGA. HERA fully saturates the available HBM bandwidth and reduces off-chip memory traffic through the proposed optimizations: First, we propose an adoption of the structural decomposition of key FHE kernels to eliminate irregular and strided HBM accesses and simplify on-chip vector permutation complexity. Second, with contiguous unit-stride HBM accesses, we introduce a novel data layout that enables fully independent access per channel. This eliminates cross-channel communication and the overhead of the built-in crossbar. Third, coupled with our structural decomposition, we develop a Shared Permutation Unit (SPU) that is flexible to handle various vector permutations in FHE with minimal hardware costs, achieving both performance and resource efficiency. Fourth, to reduce HBM traffic, we further propose Hierarchical Memory-centric Optimizations (HMOs). HMOs fuse computation across the FHE stack and maximize data reuse within the FPGA's limited SRAM capacity. Implemented on an Alveo U280, HERA outperforms all prior FPGA works on bootstrapping, a crucial FHE benchmark, achieving 1.2–4.1× speedup. Moreover, HERA delivers an average of 70.9× and 11.4× higher performance density (i.e., throughput per multiplier) than state-of-the-art ASIC-based solutions on two widely adopted applications: secure logistic regression training and encrypted image classification with ResNet-20, respectively.

CCS Concepts

• **Hardware** → **Hardware accelerators**; • **Computer systems organization** → **Reconfigurable computing**; • **Security and privacy** → **Cryptography**.

Keywords

Fully Homomorphic Encryption, CKKS, FPGA Accelerator

ACM Reference Format:

Zhihan Xu, Rajgopal Kannan, and Viktor Prasanna. 2026. HERA: A Bandwidth-efficient Accelerator for Fully Homomorphic Encryption on HBM-enabled FPGA. In *Proceedings of the 2026 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA '26)*, February 22–24, 2026, Seaside, CA, USA. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3748173.3779201>

1 Introduction

Fully Homomorphic Encryption (FHE) enables privacy-preserving computation in untrusted environments, such as cloud platforms, by allowing arbitrary computations to be performed directly on encrypted data. In a typical FHE workflow, users encrypt their data locally, outsource computation to the cloud, and decrypt the results locally using their private keys. Throughout the process, the sensitive data remains "available but invisible" to the cloud service provider [45]. This capability has made FHE an attractive foundation for a variety of applications, such as privacy-preserving machine learning [19, 30], secure database queries [38, 46], and confidential genomic analytics [25, 47], among others.

Despite its promise, FHE imposes enormous computation and off-chip memory overheads, still several orders of magnitude slower than plaintext computation [10]. To address this challenge, hardware acceleration has become essential. Prior efforts span general-purpose CPUs/GPUs [6, 15, 21, 36], as well as domain-specific FPGA and ASIC designs [3, 23, 26, 42]. CPU/GPU-based approaches [16, 21, 31] deliver limited performance due to their cache-based memory hierarchy, lack of explicit data-movement control, and inefficiencies in performing irregular vector permutations needed in FHE. State-of-the-art (SOTA) ASIC accelerators deliver up to two orders of magnitude speedup over GPUs [20, 23]. However, these ASIC designs typically rely on hundreds of megabytes of on-chip memory and thousands of specialized processing elements, sacrificing resource efficiency and consuming large silicon area and power. They also incur high non-recurring engineering costs and lack adaptability to evolving FHE techniques and applications, which limits their practicality for commercial deployment at present.

FPGAs offer a more balanced alternative, combining reconfigurability with customizability. Modern FPGA platforms also integrate HBM to address the bandwidth limitation of DRAM. With more constrained on-chip resources than ASICs, SOTA FPGA-based accelerators [3, 20, 42, 45] have focused on improving resource efficiency through logic reuse. However, their performance remains limited by the small SRAM capacity. Prior designs employ limited scheduling and on-chip data reuse strategies, resulting in excessive off-chip memory (i.e., HBM) traffic and low arithmetic intensity. Furthermore, previous designs fail to explore and exploit bandwidth utilization. These limitations are particularly pronounced in

ACM acknowledges that this contribution was authored or co-authored by an employee, contractor, or affiliate of the United States government. As such, the United States government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for government purposes only. Request permissions from owner/author(s).

FPGA '26, Seaside, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2079-6/2026/02

<https://doi.org/10.1145/3748173.3779201>

bootstrapping, the most expensive FHE workload, where long operation chains exacerbate memory bottlenecks. In addition, FPGA designs lag behind ASICs in algorithmic optimizations. ASIC designs [23, 24, 34] already exploit decomposition of the Number Theoretic Transform (NTT) and Automorphism, two key FHE kernels, to break large transforms into smaller independent ones, thereby improving scalability and simplifying hardware design [12]. In contrast, FPGA accelerators have rarely explored these opportunities.

To overcome these limitations, we present HERA, a bandwidth-efficient accelerator for FHE on HBM-enabled Reconfigurable Array. HERA co-optimizes the algorithmic techniques (i.e., kernel decomposition and customized datapath) and architectural design (i.e., bandwidth and memory utilization) to maximize performance under the stringent resource constraints of an FPGA platform. Our contributions are summarized as follows:

- We develop a novel implementation strategy on FPGA to exploit the structural decomposition of key kernels (NTT and Automorphism) into small blocks, improving data locality and eliminating strided and irregular off-chip memory accesses.
- Coupled with kernel decomposition, we introduce a lightweight Shared Permutation Unit (SPU) that is flexible to handle diverse vector permutation tasks with shared logic, improving performance and resource efficiency.
- We propose Hierarchical Memory-centric Optimizations (HMOs) that fuse computation in a fine-grained manner across the entire FHE stack, reducing off-chip memory traffic and maximizing data reuse under limited SRAM capacity on FPGA.
- We design a novel data layout that keeps each HBM channel independently and locally accessed with unit stride. This eliminates inter-channel communication and bank contention, fully saturating the available HBM bandwidth.
- Implemented on Alveo U280, HERA achieves 1.2–4.1× speedup over SOTA FPGA accelerators on bootstrapping. For end-to-end applications, it delivers an average of 70.9× higher performance density (i.e., throughput per multiplier) on secure logistic regression training, and 11.4× on encrypted image classification with ResNet-20, compared to SOTA ASIC designs.

2 Background

2.1 RNS-CKKS FHE Scheme

We focus on the CKKS scheme [9], which has gained popularity for its ability to support real-number arithmetic in machine learning (ML) applications. Fig. 1(a) illustrates the CKKS computation stack, spanning from low-level subroutines to end-to-end applications. In CKKS, a message vector $\mathbf{m} \in \mathbb{C}^{N/2}$ is first scaled by a large factor Δ and then encoded into a plaintext polynomial (Pt or $\mathbf{P}$). The Pt is encrypted with a secret key into a ciphertext (Ct), denoted $\llbracket \mathbf{m} \rrbracket := (c_0, c_1)$, which consists of two polynomials. Both Pt and Ct polynomials have degree $N - 1$. Defining $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$, a newly encrypted Ct resides in $\mathcal{R}_Q^2$, i.e., a pair of polynomials whose coefficients are integers modulo Q .

The modulus Q can span hundreds or thousands of bits. To enable efficient arithmetic on such large coefficients, the Residual Number System (RNS) is used. Specifically, the modulus is represented as $Q = \prod_{i=0}^L q_i$, where each q_i is a small prime that fits within a machine word. A polynomial in $\mathcal{R}_Q$ is thus decomposed into $L + 1$

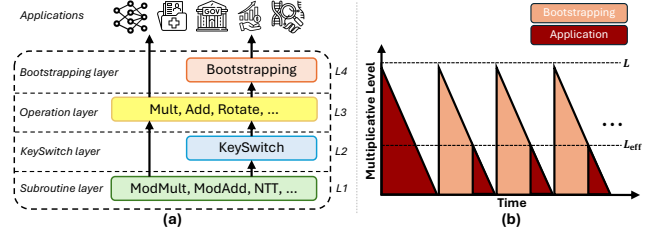


Figure 1: (a) CKKS computation stack; (b) Ct level during computation. Here, application runtime refers to the effective computation time, excluding the bootstrapping overhead.

Table 1: RNS-CKKS parameters

Param.	Description
N	# of coefficients in a Ct/Pt polynomial; a power-of-2
L	Maximum multiplicative level of Ct
ℓ	Current remaining level of Ct
Q or Q_L	$= \prod_{i=0}^L q_i$, Initial modulus of Ct
Q_ℓ	$= \prod_{i=0}^\ell q_i$, Current modulus of Ct at level ℓ
P	$= \prod_{i=0}^{k-1} p_i$, Auxiliary modulus for KeySwitch
k	# of moduli in P ; Auxiliary levels for KeySwitch
$dnum$	KeySwitch decomposition number; Max # of digits
α	$= \lceil (L + 1)/dnum \rceil$, # of limbs in one digit
β	$= \lceil (\ell + 1)/\alpha \rceil$, # of digits in Ct
L_{boot}	# of levels consumed by bootstrapping
L_{eff}	$= L - L_{boot}$, Achievable level after bootstrapping

residual polynomials, or *limbs*, with coefficients in $\mathbb{Z}_{q_i}$. A newly encrypted Ct has an initial level of L with $2(L + 1)$ limbs. Fig. 2(a) shows the RNS representation of a single Ct polynomial. The CKKS with RNS is referred to as RNS-CKKS [8]. Key parameters of RNS-CKKS are summarized in Table 1.

2.1.1 Operation Layer. CKKS mainly supports the following homomorphic operations based on modular arithmetic, where $\odot$ denotes the element-wise multiplication.

- $PAdd(\llbracket \mathbf{m} \rrbracket, \mathbf{P}') \rightarrow \llbracket \mathbf{m} + \mathbf{m}' \rrbracket = (c_0 + P', c_1 + P')$
- $PMult(\llbracket \mathbf{m} \rrbracket, \mathbf{P}') \rightarrow \llbracket \mathbf{m} \odot \mathbf{m}' \rrbracket = (c_0 \odot P', c_1 \odot P')$
- $Add(\llbracket \mathbf{m} \rrbracket, \llbracket \mathbf{m}' \rrbracket) \rightarrow \llbracket \mathbf{m} + \mathbf{m}' \rrbracket = (c_0 + c'_0, c_1 + c'_1)$
- $Mult(\llbracket \mathbf{m} \rrbracket, \llbracket \mathbf{m}' \rrbracket) \rightarrow \llbracket \mathbf{m} \odot \mathbf{m}' \rrbracket = (c_0 \odot c'_0, c_0 \odot c'_1 + c_1 \odot c'_0) + \text{KeySwitch}(c_1 \odot c'_1, \mathbf{ksk}_{mult})$
- $Rotate(\llbracket \mathbf{m} \rrbracket, d) \rightarrow \llbracket \mathbf{m} \ll d \rrbracket = \text{KeySwitch}(\psi_d(c_1), \mathbf{ksk}_{rot}^{(d)}) + (\psi_d(c_0), 0)$, where ψ_d is the automorphism on a polynomial.
- $Rescale(\llbracket \mathbf{m} \rrbracket) \rightarrow \llbracket \Delta^{-1} \odot \mathbf{m} \rrbracket = (\Delta^{-1} \odot c_0, \Delta^{-1} \odot c_1)$

Most operations only require vectorized modular multiplication (ModMult) and addition (ModAdd). Ct-Ct multiplication (Mult) and Ct rotation (Rotate) additionally require KeySwitch.

2.1.2 KeySwitch and Subroutine Layers. During Mult and Rotate, the underlying secret key changes, requiring KeySwitch to restore it and ensure decryptability. Owing to its complexity and frequent use, KeySwitch dominates FHE workloads, often accounting for most of the runtime [14, 21, 24]. As shown in Algo. 1, it relies on a key-switching key ($\mathbf{ksk}$), i.e., a $2 \times \beta$ matrix of polynomials modulo PQ_ℓ . The key step in KeySwitch is an inner product (KSKInnerProd) between a Ct polynomial (decomposed into β digits) and $\mathbf{ksk}$. Since

Algorithm 1 KeySwitch(c_1, ksk) $\rightarrow (c_0, c_1^{\text{new}})$

Input: Ct polynomial: $c_1 \in \mathcal{R}_{Q_\ell}$, Key-switching key: $\text{ksk} \in \mathcal{R}_{PQ_\ell}^{2 \times \beta}$
Output: Two Ct polynomials $(c_0, c_1^{\text{new}}) \in \mathcal{R}_{Q_\ell}^2$

- 1: $(\hat{d}_j)_{j \in [0, \beta)} \leftarrow c_1$ $\triangleright$ Decompose into β digits
- 2: $(\hat{d}_j)_{j \in [0, \beta)} \leftarrow \text{ModUp}(\hat{d}_j)$ **for** $j \in [0, \beta)$ $\triangleright \mathcal{R}_{PQ_\ell}^\beta$
- 3: $(\hat{c}_0, \hat{c}_1) \leftarrow (0, 0)$ $\triangleright \mathcal{R}_{PQ_\ell}^2$
- 4: **for** $j = 0$ to $\beta - 1$ **do**
- 5: $(\hat{k}_0, \hat{k}_1) \leftarrow \text{ksk}[j]$ $\triangleright j$ -th column of ksk , in $\mathcal{R}_{PQ_\ell}^2$
- 6: $(\hat{c}_0, \hat{c}_1) \leftarrow (\hat{c}_0 + \hat{d}_j \odot \hat{k}_0, \hat{c}_1 + \hat{d}_j \odot \hat{k}_1)$ $\triangleright \text{KSKInnerProd}$
- 7: **end for**
- 8: $(c_0, c_1^{\text{new}}) \leftarrow \text{ModDown}(\hat{c}_0), \text{ModDown}(\hat{c}_1)$ $\triangleright \mathcal{R}_{Q_\ell}^2$

the Ct polynomial resides in modulo Q_ℓ , ModUp and ModDown are needed to adjust the modulus: ModUp expands each digit from α to $\ell + k + 1$ limbs, while ModDown reduces it back to $\ell + 1$. Fig. 4(a) illustrates this dataflow.

Fig. 2(b) shows CKKS subroutines and their access patterns. Both ModUp and ModDown require RNS Base Conversion (BConv) to change the modulus. Eq. 1 shows the BConv changing modulus of a Ct polynomial from Q_ℓ to P . It outputs k new or auxiliary limbs.

$$\text{BConv}_{Q_\ell \rightarrow P}(c) = \left\{ \sum_{i=0}^{\ell} (c^{(i)} \odot \hat{q}_i^{-1} \bmod q_i) \odot \hat{q}_i \bmod p_j \right\}_{0 \leq j < k} \quad (1)$$

where $c^{(i)}$ is the i -th limb, $\hat{q}_i = \prod_{i' \neq i} q_{i'}$, and $\hat{q}_i \cdot \hat{q}_i^{-1} \equiv 1 \bmod q_j$. The first step is scaling i -th limb with $\hat{q}_i^{-1}$, which is simply a vector ModMult. The second step requires coefficient-wise access as it accumulates on the same coefficient location from different limbs.

NTT is a specialized form of the Discrete Fourier Transform (DFT), reducing the complexity of polynomial multiplication from $O(N^2)$ to $O(N \log N)$. Polynomials are transformed to the evaluation domain, where multiplication becomes element-wise. However, BConv cannot be performed in the evaluation domain, so iNTT is performed before BConv and NTT is performed afterward.

Automorphism (Auto) is a polynomial (i.e., vector) permutation on N elements. It is defined by the index mapping $\psi_d(i) = i \cdot 5^d \bmod N$, where $i \in [0, N)$, i.e., the element of index i goes to the new position at index $\psi_d(i)$.

2.1.3 Multiplicative Level and Rescale. Each multiplication (i.e., Mult, PMult) doubles the Ct scale (Δ^2). To restore the scale, Rescale is performed by transforming $\llbracket \mathbf{m} \rrbracket$ in $\mathcal{R}_{Q_\ell}$ into $\llbracket q_\ell^{-1} \odot \mathbf{m} \rrbracket$ in $\mathcal{R}_{Q_{\ell-1}}$ [8]. It is usually implemented as a specialized ModDown, reducing modulus from Q_ℓ to $Q_{\ell-1}$ by discarding the last prime modulus q_ℓ . Therefore, Rescale consumes one Ct multiplicative level, and a Ct at level ℓ can support at most ℓ multiplications.

2.1.4 Bootstrapping Layer: As shown in Fig. 1(b), Ct levels decrease with consecutive multiplications until reaching zero, leaving only a single limb with modulus q_0 is left in the Ct polynomial. To enable unlimited computation, bootstrapping restores Ct levels. It is worth noting that since bootstrapping itself consumes Ct levels, it can only achieve L_{eff} levels instead of the maximum L . Bootstrapping is known as the most expensive workload in FHE applications, which comprises a complex sequence of operations. As it comprises numerous Rotate and Mult, KeySwitch can take up to 80% of the total

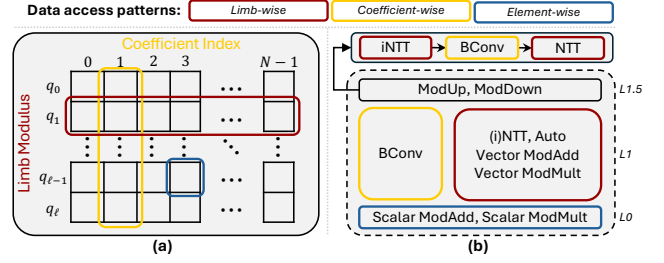


Figure 2: (a) A Ct polynomial in $\mathcal{R}_{Q_\ell}$ with RNS; (b) CKKS subroutines and the corresponding data access patterns

Table 2: NTT and Automorphism design comparison

Work	Decompose NTT	Decompose Auto	w/o Dedicated NTT Units	Unified Perm. Design
F1* [33]	✓	✓	✗	✗
CraterLake* [34]	✓	✓	✗	✗
BTS* [26]	✓	✓	✗	✗
ARK* [24]	✓	✓	✗	✗
SHARP* [23]	✓	✓	✗	✗
FAST-I* [14]	✓	✗	✗	✗
FAB† [3]	✗	✗	✓	✗
Poseidon† [45]	✗	✓	✗	✗
EFFACT†* [20]	✗	✓	✗	✗
FAST-II† [42]	✗	✗	✓	✓
OLA† [43]	✗	✗	✓	✓
HERA†	✓	✓	✓	✓

Note: Both [14] and [42] are named FAST, and we use I and II to distinguish between them.
 *ASIC-based accelerator; †FPGA-based accelerator

runtime [14]. This motivates the need for a **hierarchical optimization strategy applied across the full CKKS stack to obtain maximum and combination performance improvements.**

2.2 Prior CKKS Accelerators

Prior ASIC-based accelerators [14, 23, 24, 26, 33, 34] implement dedicated hardware units for different subroutines like NTT unit and Auto unit. They also employ large on-chip memory to buffer large working sets during bootstrapping and other benchmarks. While this achieves high throughput, it comes at the cost of a large silicon area and poor overall resource utilization. By contrast, FPGA accelerators [3, 42, 43] are resource-constrained and thus mainly adopt a resource-sharing architecture, where modular arithmetic units are reused for different subroutines. This improves resource utilization but still poses challenges under constrained SRAM and bandwidth, particularly for subroutines with irregular access patterns and limited locality.

Among the subroutines, NTT and Auto exhibit inter-element dependencies within a limb, resulting in strided or irregular memory accesses that degrade bandwidth efficiency. Prior ASIC designs address this with structural decomposition, breaking large transforms into smaller independent ones to improve locality and simplify permutations. For example, instead of a monolithic N -point NTT, they deploy multiple fully-pipelined small-point NTT units to sustain high performance. In contrast, most FPGA-based accelerators adopt iterative NTT implementations [29] due to compute resource sharing. In such designs, later stages cannot begin until earlier ones are completed, which requires explicit handling of inter-stage permutations. FAB [3], Poseidon [45], and EFFACT [20] perform memory-based permutation, writing intermediate results back to

memory to align inputs for subsequent stages. Poseidon further employs a high-radix NTT (radix > 2) core to improve locality and reduce permutation overhead. Other works, such as FAST-II [42] and OLA [43], introduce hardware-based permutation units for streaming between stages; their permutation unit can also be reused for Auto, offering a unified design. For Auto specifically, Poseidon applies structural decomposition for a hardware-friendly implementation, while FAB relies on large Register Files (RFs) to map indices due to the high polynomial degree in FHE. Table 2 summarizes the design strategies for NTT and Auto across prior accelerators.

2.2.1 NTT Decomposition: A large N -point NTT can be recursively decomposed into multiple smaller dp -point NTTs that better align with hardware resources [39]. If the hardware can efficiently process dp -point transforms, then an N -point NTT can be expressed in $\lceil \log N / \log dp \rceil$ dimensions. In each dimension, all dp -point transforms are computed independently. Between dimensions, the data is reorganized by a transpose, and twiddle factor multiplication is required. For example, in a 2D decomposition (i.e., the four-step NTT [5, 37]) with $N = R \times C$, we first perform R transforms of length C . After element-wise twiddle multiplications and a matrix transpose, C transforms of length R are executed.

2.2.2 Automorphism Decomposition: Like NTT, an Auto can also be decomposed into smaller permutations. Consider a polynomial of size $N = R \times C$ arranged in a row-major matrix. The new column index c' of element i depends only on its original column index ($c = i \bmod C$), as $c' = \psi_d(i) \bmod C = i \times 5^d \bmod C = c \times 5^d \bmod C$ only depends on c . This property ensures that elements within a column remain in the same column after permutation [12, 45]. Therefore, Auto can be computed column by column: each column is permuted independently and then written back to the appropriate column position. Fig. 3(b) shows an example of 8-point Auto, where elements in the same column stay grouped. The new row and column indices are given by [12, 45]:

$$r' = r \times 5^d \bmod R + \lfloor \frac{c \times 5^d}{C} \rfloor \bmod C \quad (2)$$

$$c' = c \times 5^d \bmod C \quad (3)$$

3 HERA Optimizations

3.1 NTT and Automorphism Decomposition

As shown in Table 2, HERA is the first FPGA-based accelerator to systematically exploit kernel decomposition for both NTT and Auto. Decomposition mitigates the inefficiencies of irregular off-chip accesses and the complexity of on-chip permutations [12].

For NTT, coefficient access follows a strided pattern within a polynomial. When N is large, directly fetching these strided inputs from off-chip memory leads to non-contiguous accesses, poor spatial locality, and underutilized bandwidth. For Auto, the access is more challenging: the permutation corresponds to an irregular index mapping that scatters coefficients across memory, forcing prior FPGA designs without decomposition to use large RFs or buffers to hold the entire polynomial during permutation.

By decomposing both transforms into smaller contiguous blocks, HERA eliminates strided off-chip accesses in NTT and irregular accesses in Auto. This not only improves bandwidth efficiency but also

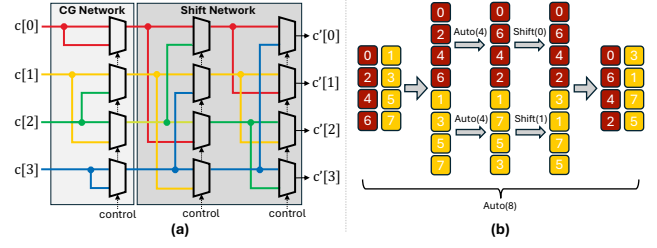


Figure 3: (a) Shared Permutation Unit (SPU) in HERA, e.g., $dp = 4$, cascading a Constant-Geometry (CG) network and a shift network; (b) Length-8 Auto with decomposition

simplifies permutation logic and routing, since each independent block involves fewer data dependencies. Discussed in the Sec. 3.2, this decomposition further enables our lightweight Shared Permutation Unit (SPU), which offers an efficient and unified hardware solution for permutations in both NTT and Auto.

3.2 Shared Permutation Unit (SPU)

As shown in Table 2, HERA adopts a SPU for both NTT and Auto, improving resource efficiency. Prior works such as FAST-II [42] and OLA [43] were the first to propose unified permutation designs for FHE. However, their solutions incur long latency and significant memory overhead. Specifically, a dp -lane of their permutation unit requires $2 \log dp + N/dp$ latency cycles and dp buffers of size N/dp each to permute a polynomial of length N . In contrast, HERA leverages a much simpler SPU based on the design in [12], which is coupled with the structural decomposition of both NTT and Auto. As shown in Fig. 3(a), the dp -lane SPU in HERA consists of two components: a single-stage Constant-Geometry (CG) network and a shift network with $\log dp$ stages.

The CG network handles inter-stage permutations for NTT. Since decomposition reduces the transform size to dp points, only a single CG stage is needed to complete each permutation. For Auto, the shift network performs cyclic shifts derived from decomposition. As shown in Eq. 2, the second term is independent of the row index r , and thus a constant for each specific column, representing a cyclic shift. Consequently, a large Auto can be recursively decomposed into smaller Autos and shifts. At the lowest level (length-2), Auto degenerates to the identity mapping, leaving only shift operations. These shifts can then be merged, so any dp -point Auto passes through the SPU exactly once. Fig. 3(b) illustrates this process for an 8-point Auto, which decomposes into two 4-point permutations and shifts. The 4-point Auto can be further decomposed.

With NTT decomposition, a matrix transpose of the polynomial tensor is required when switching dimensions. This can be achieved using a combination of the CG network and the shift network. Intuitively, multiple shifts and write operations can realize most of the transpose, while the CG network is enabled when previously adjacent coefficients become separated and interleaved after a dimension switch. A detailed mapping explanation is provided in [12]. The original design in [12] uses two CG stages to implement the transpose. In our implementation, we eliminate one of two stages, thereby reducing redundancy and hardware overhead, as further discussed in Sec. 4.2.2.

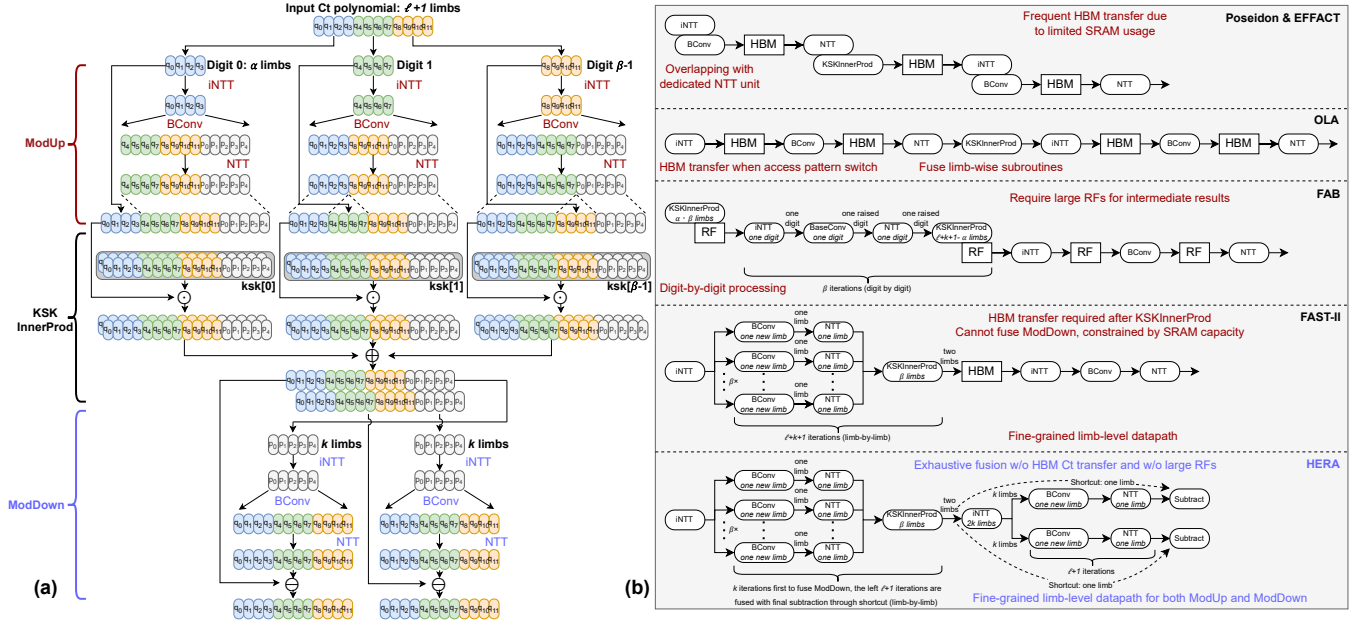


Figure 4: (a) KeySwitch dataflow diagram (e.g., $\ell = 11, \beta = 3, \alpha = 4, k = 5$); (b) Customized KeySwitch datapath comparison. HERA effectively avoids intermediate Ct transfer to HBM without the usage of large RFs.

3.3 Hierarchical Memory-centric Optimizations

Due to the large working sets of FHE, memory efficiency and on-chip data reuse are critical for performance. Prior ASIC designs typically rely on large SRAM buffers; for example, the SOTA FAST-I [14] employs 245 MB of on-chip memory. By contrast, FPGAs are much more resource-constrained, with only about 43 MB of SRAM available on the target AMD Alveo U280 device. As a result, memory-centric optimizations tailored for FPGAs are essential for practical FHE acceleration.

Several FPGA accelerators have explored memory-aware techniques to reduce off-chip data traffic. FAB [3] modifies the KeySwitch datapath to avoid reading and writing Ct limbs to off-chip memory. FAST-II [42] optimizes bootstrapping by fusing subroutines in its linear transformations, while OLA [43] applies compiler-level fusion strategies. However, these designs suffer from one or more of the following limitations: (i) focusing only on a single workload or layer rather than the full FHE stack, which limits overall system performance; (ii) relying on large RFs to compensate for limited SRAM, resulting in high logic utilization; or (iii) employed fusion strategies cannot fully exploit data reuse opportunities under the constrained SRAM. To overcome these challenges, we propose Hierarchical Memory-centric Optimizations (HMOs) that adopt a holistic view of the FHE stack, summarized as ①–⑥. HMOs systematically maximize on-chip data reuse under the FPGA’s limited SRAM capacity, while minimizing costly off-chip data transfers. The hierarchical optimizations across the stack are detailed as follows.

3.3.1 Subroutine Layer. All FHE workloads are composed of subroutines. If each subroutine is expressed in limb-wise access, they can be fused to process Ct limb by limb, substantially reducing on-chip memory requirements. However, BConv requires orthogonal coefficient-wise access, which prevents direct fusion of the common

workload pattern $\text{iNTT} - \text{BConv} - \text{NTT}$ used in modulus switch (i.e., ModUp , ModDown). Without careful use of on-chip memory, this leads to additional off-chip memory traffic and bandwidth inefficiency. The problem is particularly severe for practical parameter sets achieving 128-bit security¹, where N is typically 2^{16} or 2^{17} and Q exceeds one thousand bits. With parameters in Table 3, a Ct polynomial requires about 10 MB of storage. For prior FPGA designs with on-chip memory smaller than the size of one Ct polynomial, such as Poseidon [45] and EFFACT [20], Ct must be transferred to off-chip memory during modulus switch. Moreover, when a Ct stored in limb-wise format is accessed coefficient-wise, off-chip reads become strided, resulting in poor bandwidth utilization [44].

To overcome these issues, we propose two optimizations: ① **Perform modulus switch on-chip.** Executing BConv entirely on-chip avoids coefficient-wise off-chip accesses and keeps all off-chip transfers consecutive in limb-wise pattern, thereby fully exploiting HBM bandwidth. This requires sufficient on-chip memory to hold at least one Ct polynomial. ② **Fuse BConv and NTT through a single-limb datapath.** A major challenge is that BConv produces multiple auxiliary limbs during modulus switch. For example, with parameters in Table 3, after ModUp a Ct in $\mathcal{R}_{PQ}^\beta$ (Line 2, Algo. 1) exceeds 40 MB. Storing all auxiliary limbs on-chip is impractical. Instead, we perform NTT immediately as each new or auxiliary limb is generated by BConv , rather than waiting for all auxiliary limbs to be produced. This fuses BConv and its following NTT despite their different access patterns.

Our ② extends one prior optimization, i.e., on-the-fly limb generation, in FAST-II [42], which applied this idea only to ModUp processing of β digits. Specifically, as illustrated in HERA’s datapath in Fig. 4(b), for each digit, BConv generates one new limb that

¹The best-known attack requires $\geq 2^{128}$ operations to break the scheme.

is immediately sent to NTT. We extend this approach to ModDown and hierarchically integrate it with ModUp for KeySwitch, as detailed in the following Sec. 3.3.2.

3.3.2 KeySwitch Layer. For the KeySwitch datapath, we propose to ③ **exhaustively fuse subroutines with single-limb dataflow**. Building on ②, which enabled fusion of BConv despite its orthogonal access pattern, we extend single-limb dataflow to the remaining limb-wise subroutines. This allows fine-grained on-chip data reuse at the limb level, avoiding the need to buffer entire Ct polynomials and thereby reducing both SRAM usage and off-chip traffic. Specifically, after BConv generates β new limbs and they are transformed by NTT, these limbs can directly proceed to KSKInnerProd, as shown in HERA's datapath in Fig. 4(b). It is because these β limbs are mutual operands, which share the same modulus.

A key bottleneck in prior FPGA designs is handling the increased size of Ct during KeySwitch. Specifically, the two polynomials ($\hat{c}_0, \hat{c}_1$) after KSKInnerProd (Line 6, Algo. 1) occupy about 27 MB with the parameters in Table 3. Most designs must write these results back to off-chip memory. Even with single-limb dataflow (as in FAST-II [42]), keeping the full KSKInnerProd result on-chip is challenging, since the input polynomial itself already consumes about 10 MB. Thus, intermediate results are typically written to HBM limb by limb. To overcome this limitation and effectively fuse KSKInnerProd with the following ModDown, we introduce ④ **selective k -limb computation**: KSKInnerProd first computes the auxiliary k limbs in P required by BConv of ModDown, and then computes the remaining $\ell + 1$ limbs. As illustrated in HERA's datapath in Fig. 4(b), each newly generated limb in modulus Q_ℓ from KSKInnerProd is fused with the new limb generated by BConv through a shortcut. This achieves single-limb dataflow in ModDown and tightly fuses KSKInnerProd with ModDown, realizing exhaustive algorithmic fusion across the KeySwitch datapath.

As KeySwitch dominates FHE workloads, several FPGA accelerators have targeted this datapath. Fig. 4(b) compares HERA's design against prior works. Poseidon [45] and EFFACT [20] frequently communicate with HBM during modulus switching due to limited SRAM. OLA [43] cannot fuse BaseConv and NTT, which require different access patterns. FAB [3] processes digit-by-digit but depends on large RFs to avoid off-chip Ct transfers. FAST-II [42] achieves ① and ② during ModUp, but still writes KSKInnerProd results back to HBM and fails to fuse with ModDown. In contrast, HERA performs the entire KeySwitch datapath on-chip with exhaustive fusion, without requiring additional distributed buffers.

3.3.3 Operation Layer. Among the homomorphic operations, we focus on Mult, Rotate, and Rescale, since other operations are based on vectorized ModMult or ModAdd. Both Mult and Rotate invoke KeySwitch, allowing us to hierarchically integrate our customized KeySwitch datapath with operation-level optimizations.

For Mult, a Rescale is required at the end. We extend our ④ with the operation-level optimization ⑤ **merge Rescale into ModDown during Mult** [2]. Recall that ModDown reduces the modulus by P , while Rescale reduces the modulus by Δ (approximated by the last modulus q_ℓ). Instead of performing them separately, we directly apply ModDown to reduce the modulus by $P \cdot \Delta$, effectively eliminating the standalone Rescale since it can be viewed as a special case of ModDown. With this hierarchical integration, BConv of

ModDown requires $2(k + 1)$ rather than $2k$ input limbs, and the iteration count reduces from $\ell + 1$ to ℓ . For Rotate, additional Auto is required. The overhead of Auto is relatively smaller compared with KeySwitch. However, real-world workloads such as bootstrapping typically apply many Rotate operations on the same Ct polynomial. Thus, further hierarchical optimization integration with bootstrapping is required, discussed in the following Sec. 3.3.4.

3.3.4 Bootstrapping Layer. As the most expensive workload in FHE, bootstrapping must be carefully optimized for FPGAs, which are constrained by limited on-chip memory. The dominant computation in bootstrapping is PtMatVecMul (Eq. 4), which multiplies a Ct with a Pt matrix. The matrix is a sparse DFT matrix with multiple non-zero diagonals, so PtMatVecMul is performed by the accumulation of a series of Hadamard multiplication (PMult), i.e., Diagonal InnerProd, and vector shifts (Rotate).

$$M * \llbracket \mathbf{m} \rrbracket = \sum_{i=0}^r \text{PMult}(\text{Rotate}(\llbracket \mathbf{m} \rrbracket, d_i), \text{diag}_i[M]) \quad (4)$$

When multiple Rotate operations are applied to the same Ct, an essential cryptographic technique known as ⑥ **Hoisting** [7] can be used to reduce the cost of ModUp and ModDown. Hoisting allows us to perform ModUp only once, reusing its result across all r rotations, and postpones ModDown until after all r rotations. We integrate HERA's fused KeySwitch datapath, incorporating ①②③④, with ⑥. Furthermore, since each PMult requires a Rescale, we merge ⑤ into the hoisted ModDown. In this way, bootstrapping leverages all layers of our hierarchical optimizations (①–⑥), achieving combined and cumulative improvements. The resulting PtMatVecMul datapath is illustrated in Fig. 5.

The hoisting technique has been adopted by SOTA accelerators [14, 40–42] and also appears in other application domains that require repeated Rotate operations on the same Ct, such as privacy-preserving CNN inference [13].

3.4 Uniform Data Layout and Access Pattern

Efficient use of HBM bandwidth requires alignment between the data layout in off-chip memory and the access patterns of subroutines. A mismatch leads to strided or irregular accesses that leave the available bandwidth underutilized. For example, if limbs are stored limb-wise but a subroutine such as BConv requires coefficient-wise access, fetching data incurs strided loads and degraded bandwidth efficiency. Prior work [44] addresses this by inserting a custom interconnect between the HBM controller and accelerator to dynamically switch data layouts. While effective, such solutions introduce additional hardware complexity and buffering overhead.

In HERA, kernel decomposition and memory-centric optimizations ensure that limbs are always accessed in a continuous, limb-wise, block-by-block manner. This simplifies data storage, eliminates the need for complex interconnects, and improves bandwidth utilization. Specifically, the NTT and Auto decompositions enable sequential access to small contiguous blocks, and ① of the HMOs allows BConv to be executed entirely on-chip. In contrast, designs such as Poseidon [45] and EFFACT [20], constrained by limited SRAM, cannot realize ① and therefore may resort to complex interconnect or suffer from bandwidth degradation.

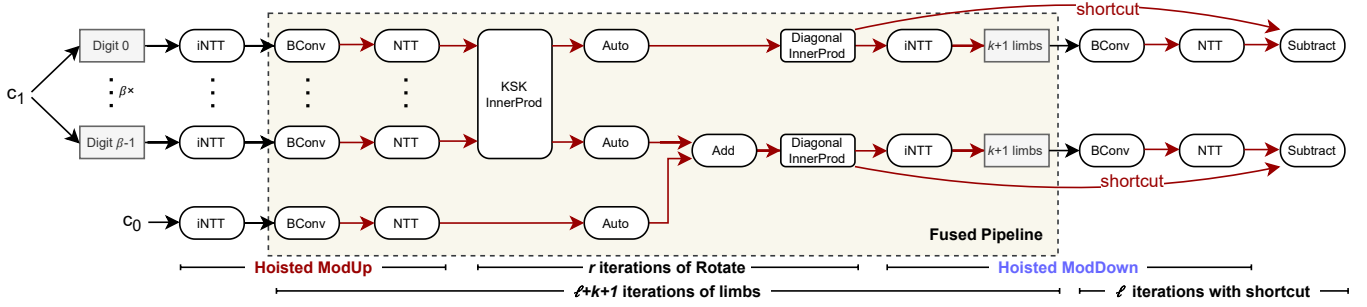


Figure 5: Proposed optimized PtMatVecMult datapath for bootstrapping, leveraging HMOs across all computation stacks, achieving maximum data reuse under the constrained SRAM. Red arrow denotes the fine-grained single-limb datapath.

Based on the block-by-block access property, we propose a uniform data layout for all polynomials. Unlike FAB [3], which allocates separate buffers for different polynomial types, HERA employs a uniform storage abstraction across all polynomials. Moreover, instead of allocating a subset of limbs to a specific HBM channel based on their prime moduli, which may lead to bandwidth under-utilization, HERA maps each limb across all HBM pseudo-channels (PCs) using block-cyclic distribution. Each PC is paired with an on-chip bank through a dedicated AXI interface. Under this scheme: (i) Every off-chip access is sequential and unit-stride; (ii) Each PC is accessed locally and independently by its paired bank, eliminating inter-channel communication and the need for a complex on-chip NoC; (iii) coefficients with the same index across different limbs are co-located in the same PC, whether the limbs come from the same polynomial (e.g., coefficient-wise BConv) or from different polynomials (e.g., limb-wise Add). This allows every subroutine to be processed directly without inter-bank data movement.

This co-design of uniform limb-wise access and data layout saturates the available HBM bandwidth, simplifies on-chip memory organization, and improves memory efficiency (i.e., utilization). The microarchitectural realization of the above idea, namely the bank group organization, is described in Sec. 4.2.3.

3.5 Summary of Optimizations

The optimizations presented in this section collectively enhance both bandwidth and memory efficiency of HERA. By adopting NTT and Auto decomposition and ①, HERA avoids strided and irregular HBM accesses, and the decomposed blocks naturally align with the proposed uniform data layout. The uniform block-cyclic data layout guarantees full and balanced utilization of all HBM channels. The decomposition further enables the SPU, which shares logic across permutation tasks, improving both resource efficiency and permutation performance. In parallel, the proposed HMOs exploit on-chip data reuse under the limited SRAM capacity of FPGAs, exhaustively fusing subroutines into fine-grained limb-level pipelines and reducing costly intermediate Ct transfers.

Together, these optimizations saturate available bandwidth, minimize redundant HBM traffic, and demonstrate the effective use of scarce on-chip memory on FPGAs. More importantly, they highlight that holistic algorithm–architecture co-design is essential for improving bandwidth and memory efficiency, unlocking the full potential of FPGAs for full-stack FHE acceleration.

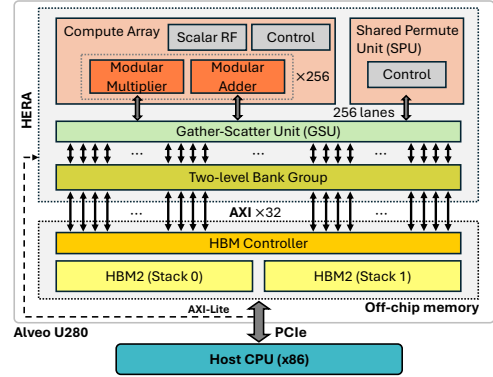


Figure 6: System architecture with a host and an Alveo U280

4 HERA Architecture

4.1 Overall System Architecture

Fig. 6 illustrates the overall system, consisting of a host CPU (x86) and an HBM-enabled FPGA. The host communicates with the FPGA over a PCIe interface, which enables data transfers between the host and the HBM stacks on the device. The two HBM stacks provide a peak bandwidth of up to 460 GB/s. HERA employs shared compute logic, a unified data layout and access pattern, and a unified permutation design. Therefore, its architecture remains simple, comprising a compute array, a Shared Permutation Unit (SPU), a Gather-Scatter Unit (GSU), and an on-chip memory bank group.

Execution begins with the host launching HERA via AXI-Lite control registers. Input limbs are transferred from the host to HBM channels through PCIe. During computation, data streams from the HBM stacks into the on-chip bank group via 32 AXI interfaces, each operating independently with its designated bank. Intermediate data is exchanged between the on-chip bank group and HBM, while only the final outputs are returned to the host.

4.2 Microarchitecture

The number of lanes or degree of parallelism (dp) in HERA is set to 256, determined by resource availability, operation efficiency, and control complexity. First, modular multiplication is costly in terms of DSP usage; thus, dp is constrained on FPGA. Second, for a practical and bootstrappable FHE system targeting 128-bit security, the polynomial length N must be at least 2^{16} . Choosing $dp = \sqrt{N}$

enables a two-dimensional decomposition of NTT, which significantly simplifies twiddle-factor management and index mapping. Consequently, each NTT requires only one matrix transpose and one inter-dimension twiddle multiplication. This choice also eliminates one CG network in the SPU compared with the original design [12]. The same complexity reduction applies to Auto when $dp = \sqrt{N}$. Finally, setting $dp = \sqrt{N}$ avoids inter-bank communication: each length-256 vector is independent and evenly distributed across all banks, ensuring a uniform data layout and access pattern.

4.2.1 Compute Array. The compute array consists primarily of dp modular multipliers and modular adders. A scalar RF stores preliminary parameters such as the prime moduli chain. The modular adder is configurable via a mode bit to perform either addition or subtraction. All FHE workloads are executed on the compute array by expressing them as sequences of length- dp ModMult and ModAdd operations. For instance, the compute array supports both Decimation-In-Time (DIT) and Decimation-In-Frequency (DIF) butterfly operations for NTT and iNTT, depending on the order of ModMult and ModAdd. Similarly, BConv can be expressed as ModMult followed by ModAdd, thereby realizing a multiply-accumulate datapath.

The modular multiplier is critical to both resource usage and overall performance of the accelerator. We adopt the pipelined Barrett Modular Multiplier (BMM) design from [27], which is more resource-efficient than AMD’s built-in multiplier IP. Each BMM requires three multipliers. With a chosen coefficient size of $\log q = 50$ bits, the implementation uses one 50-bit multiplier and two 51-bit multipliers. Selecting 50-bit coefficients achieves efficient use of DSP resources. A 51-bit input is partitioned into either three 17-bit chunks or a 26-bit and 25-bit pair, mapping onto the 27×18 multiplier of a DSP slice. As a result, each 51-bit multiplier consumes 6 DSPs, and the 50-bit multiplier has a similar pattern. Moreover, with $\log q = 50$, one of the 51-bit multipliers that only needs the lower half of the result (51 bits out of 102 bits) can save one DSP, consuming only 5 slices.

4.2.2 Shared Permutation Unit (SPU). For $dp = 256$, the SPU comprises a single CG stage followed by eight shift-network stages. As a result, each length-256 sub-transform can be permuted in only 9 cycles. The SPU is fully pipelined, sustaining a throughput of 256 points per cycle. In our configuration ($\log N \bmod \log dp = 0$), we identified that one CG stage can be eliminated compared to the original design [12]. The prior design employs a CG stage cascaded with an inverse-CG stage to support all possible permutation patterns required for matrix (tensor) transposes in the NTT. With our setup, the inverse-CG stage is unnecessary, eliminating redundancy and reducing hardware cost. Furthermore, the control signals are simplified under this configuration, allowing them to be pre-generated and stored directly within the SPU with negligible overhead.

4.2.3 Bank Group Organization and Gather-Scatter Unit (GSU). As described in Sec. 3.4, HERA organizes on-chip memory into 32 banks, each directly paired with one HBM PC to ensure fully independent and local access. To evenly distribute the workloads across 32 banks, each limb is partitioned into sub-vectors of length dp , and each sub-vector is further partitioned into 32 blocks. The block size is therefore $\text{block}_{\text{size}} = dp/32 = 8$ coefficients. For a global coefficient index $i \in [0, N)$ within a limb, we define the sub-vector ID

Table 3: FHE parameter set used for evaluation

N	$\log q$	$\log p$	L	k	$dnum$	λ	L_{boot}
2^{16}	50	50	23	9	3	128	15

s and the local index j as $s = \lfloor i/dp \rfloor$ and $j = i \bmod dp$, respectively. The block ID and its corresponding bank assignment are then given by $\text{block}_{\text{ID}} = \lfloor j/\text{block}_{\text{size}} \rfloor$ and $\text{bank}_{\text{ID}} = \text{block}_{\text{ID}}$. This block-cyclic distribution ensures that every length- dp sub-vector is evenly partitioned across all 32 banks, achieving balanced utilization of all HBM PCs while preserving unit-stride access within each channel.

The Gather-Scatter Unit (GSU) serves as the interface between the bank group organization and the processing logic. During execution, it collects one block from each of the 32 banks to reconstruct a length- dp sub-vector per cycle, which is then streamed directly to the compute array or the SPU. In the reverse direction, the GSU scatters outputs back into the banks. By operating at the granularity of sub-vectors, the GSU eliminates the need for a complex NoC or interconnect to shuffle data between banks, ensuring efficient, ordered, and conflict-free data delivery.

To support concurrent memory requests from the SPU, compute array, and HBM controller, HERA employs a two-level bank group hierarchy. Each bank is subdivided into eight sub-banks, enabling parallel reads and writes. Each sub-bank is built from BRAMs or URAMs with a width equal to the block size, supporting one block read and one block write per cycle. At the system level, the external request granularity is a length- dp sub-vector, which the GSU distributes into 32 blocks across banks. Within each bank, an arbiter maps block-level requests from GSU to available sub-banks, ensuring conflict-free access. In particular, blocks belonging to the same sub-vector ID but from different limbs are interleaved across sub-banks, enabling concurrent operand access and preventing collisions where two mutual operands map to the same sub-bank.

Overall, this organization enables high-throughput memory access: the GSU provides lightweight gather-scatter functionality, and the bank hierarchy ensures sufficient bandwidth. Together, they sustain full HBM bandwidth utilization while avoiding the complexity of large crossbars or on-chip networks.

5 Evaluation

5.1 Experimental Setup

We evaluate HERA in a real-world x86 CPU system with an AMD Alveo U280 FPGA. HERA is implemented in Verilog HDL and synthesized using Vivado 2023.1. The RTL is compiled and linked into a binary executable with Vitis 2023.1. The host program, written in C++, initializes the execution and limb transfer from HBM. HERA operates at 300 MHz and adopts a 512-bit AXI port width, with each port functioning independently. During burst transfers of limbs from HBM, the design achieves approximately 13 GB/s per port.

Table 3 summarizes the FHE parameter set used by HERA, which provides 128-bit security (λ). As discussed earlier, we choose the size of each residual modulus as $\log q = 50$ bits to optimize DSP utilization with dp -lanes under limited FPGA resources. The choice of $\log q$ also affects the maximum number of Ct levels. For a practical, bootstrappable FHE system achieving 128-bit security, the minimum polynomial length is 2^{16} . On FPGAs with constrained

Table 4: Resource usage of HERA on Alveo U280

Resource	Available	Utilized	% Utilization
LUTs	1,304K	865K	66.4
FFs	2,607K	1,046K	40.1
DSP	9,024	4,352	48.2
BRAM	4,032	3,795	94.1
URAM	962	800	83.2

SRAM capacity, selecting $N = 2^{16}$, as in most prior FPGA-based accelerators, is a sweet spot, since it allows our proposed HMOs to maximize Ct reuse on-chip. When N scales to 2^{17} , the Ct size at least doubles; HERA still supports this setting, but only partial HMOs can be applied.

5.2 Results and Discussion

5.2.1 Resource Utilization. Table 4 shows the resource consumption of HERA. HERA consumes a large proportion of available memory blocks. Specifically, approximately 10% of the used BRAM is consumed by AXI interfaces. The remaining BRAM and all URAM are used for building our on-chip bank group, totaling 35.6 MB. Moreover, only the compute array consumes DSP resources, with one modular multiplier taking 17 DSPs.

5.2.2 Basic FHE Operations. Table 5 presents the latency comparison of HERA against prior works. Both CPU and GPU baselines [21] are widely adopted in the literature. The CPU baseline is measured using an Intel Xeon Gold 6234 with a single thread, and the GPU baseline is obtained on an NVIDIA Tesla V100, with the best latency results reported. It is worth noting that V100 offers nearly twice the peak HBM bandwidth of the Alveo U280. HEAX [32] represents the pioneering FPGA-based implementation of FHE. The other four SOTA FPGA accelerators [3, 42, 43, 45] are all evaluated on the same Alveo U280 platform as HERA.

HERA achieves 1.23–4.08× speedup over the GPU across basic FHE operations. As expected, Add and PMult are bandwidth-bound, yielding smaller gains compared to compute-bound operations such as Rescale. Relative to HEAX, HERA delivers up to a 5× speedup on Mult, primarily due to higher resource efficiency, greater off-chip bandwidth, and a larger dp . For FAB and FAST-II, HERA achieves comparable performance while using 15% fewer DSPs than FAB and 23% fewer than FAST-II. In addition, FAST-II adopts $dp = 512$, doubling the number of modular multipliers compared to HERA. Compared with Poseidon and OLA, HERA achieves more than a 2× speedup on both Mult and Rotate, primarily due to reduced HBM transfers enabled by the customized KeySwitch datapath under the proposed HMOs, as well as more efficient bandwidth utilization.

5.2.3 Benchmarks. To ensure fair comparison, benchmark evaluations must adopt practical FHE parameter sets that support bootstrapping and provide at least 128-bit security. We evaluate HERA on the following widely used benchmarks:

(1) Bootstrapping: A key benchmark for assessing practical FHE performance. Two common metrics are reported: the latency T_{boot} and the amortized time $T_{A.S.}$ [21]:

$$T_{A.S.} = \frac{T_{boot} + \sum_{i=0}^{L_{\text{eff}}} T_{\text{Mult}}(i)}{L_{\text{eff}} \cdot (N/2)}. \quad (5)$$

Table 5: Latency (ms) of basic CKKS operations compared with prior CPU, GPU, and FPGA works

Work	Freq. (GHz)	Add	Mult	PMult	Rotate	Rescale
CPU [21]	3.3	28.12	2645	26.22	2579	145
GPU [21]	1.2	0.16	2.96	0.14	2.55	0.49
HEAX [32]	0.3	0.24	8.4	0.24	-	-
FAB [3]	0.3	0.04	1.71	-	1.57	0.19
FAST-II [42]	0.3	0.09	1.42	0.08	1.36	0.14
Poseidon [45]	0.45	0.07	3.66	0.08	3.31	0.25
OLA [43]	0.3	0.15	3.92	0.15	3.75	0.39
HERA	0.3	0.13	1.68	0.09	1.58	0.13

$T_{A.S.}$ also denotes the reciprocal throughput, i.e., the average time per slot per Mult, of the CKKS system, where $T_{\text{Mult}}(i)$ denotes the Ct-Ct multiplication time at level i . Normalized by the message length and L_{eff} , $T_{A.S.}$ reflects practical bootstrapping performance in real-world applications and has become a standard metric [3, 20, 42].

(2) HELR: A logistic regression model [18, 19] that trains a binary classifier on a subset of MNIST [11]. Following prior works [3, 20, 24, 42], we train the model for 30 iterations and report the average training time per iteration. Each iteration consumes 5 Ct levels; thus, a number of bootstrappings are required during training.

(3) ResNet-20: A CNN model implemented under the CKKS [30], consisting of both bootstrapping and homomorphic convolutions. In line with prior works, we evaluate performance by measuring inference latency on a single encrypted CIFAR-10 [28] image.

Table 6 summarizes the performance and system parameters of HERA in comparison with prior works. The CPU baseline [24] uses Intel Xeon Platinum 8358 (32-core) with the Lattigo FHE library [31]. ASIC-based accelerators typically leverage a large number of multipliers and a substantial SRAM capacity, and assume up to 1 TB/s of HBM bandwidth. By contrast, HERA employs only 256 modular multipliers. The SRAM usage of HERA is 35.6 MB, demonstrating effective use of constrained FPGA resources through HMOs.

HERA achieves 500–970× speedup across all benchmarks compared to the CPU baseline. Against the GPU, HERA is about 8× faster on bootstrapping latency and 13× faster on HELR. To the best of our knowledge, HERA delivers the lowest reported bootstrapping latency on a single FPGA board, achieving a 1.20–4.13× speedup over SOTA FPGA accelerators. These gains stem primarily from improved bandwidth efficiency and the proposed PtMatVecMul datapath that further reduces Ct transfers via HBM. In terms of amortized time ($T_{A.S.}$), HERA is 2.71× and 3.22× faster than FAB and EFFACT, respectively. For HELR, HERA outperforms FAB, Poseidon, and EFFACT by 1.73×, 1.23×, and 1.09×. For ResNet-20, HERA achieves respective 1.23× and 1.52× higher performance than Poseidon and OLA, and is marginally faster than EFFACT. Despite not demonstrating the optimal single-operation latency in Table 5, HERA delivers superior performance on deep benchmarks by leveraging holistic optimizations across operations.

HERA, FAST-II, and FAB explore different points in the design space under the same HE parameter set ($\log Q$, $\log P$, N , d_{num}). FAST-II adopts a dp that is twice as large as HERA’s, together with a larger Ct level, which reduces the number of required bootstrappings and emphasizes raw throughput. HERA adopts $dp = \sqrt{N}$,

Table 6: System parameters and performance comparison with prior works

	Lattigo (CPU) [24, 31]	Over 100× (GPU) [21]	BTS (ASIC) [26]	ARK (ASIC) [24]	SHARP (ASIC) [23]	FAST-I (ASIC) [14]	FAB (FPGA) [3]	Poseidon (FPGA) [45]	EFFACT (FPGA) [20]	FAST-II (FPGA) [42]	OLA (FPGA) [43]	HERA (FPGA) -
Frequency (GHz)	3.4	1.2	1.0	1.0	1.0	1.0	0.3	0.45	0.3	0.3	0.3	0.3
# of lanes (dp)	-	-	2048	1024	1024	1024	256	512	256	512	512	256
# of multipliers	-	-	8192	20480	35776	48144	256	512	512	512	512	256
HBM bandwidth (GB/s)	-	900	1024	1024	1024	1024	460	460	460	460	460	460
SRAM capacity (MB)	43	36	512	588	198	281	42.2	8.6	7.6	33.8	22.5	35.6
T_{boot} (ms)	3.90e4	328	22.9	3.52	3.12	1.38	92.4	127	-	48.1	166	40.2
$T_{A.S.}$ (ns)	88.0K	716	45.7	14.3	12.8	5.40	477	-	566	81.0	-	176
HELRL (ms)	37.1K	775	28.4	7.42	2.53	1.33	103	73.0	64.6	36.0	-	59.4
ResNet-20 (s)	1377	-	2.020	0.125	0.099	0.060	-	2.661	2.175	1.859	3.300	2.170

featuring a structurally balanced architecture with low design complexity (e.g., kernel decomposition and memory coalescing) and high resource efficiency. Compared with FAB, HERA employs a slightly smaller $\log q$ for DSP packing efficiency.

For ASIC designs, rapid progress over the past years has yielded significantly higher performance than FPGAs, enabled by the use of more modular multipliers, higher operating frequencies, greater HBM bandwidth, and larger SRAM capacity. These advantages allow ASICs to excel on complex workloads such as ResNet-20, where large SRAM capacity supports the concurrent storage of multiple intermediate ciphertexts. However, HERA achieves much higher resource efficiency compared to ASICs, as detailed in Sec. 5.2.4.

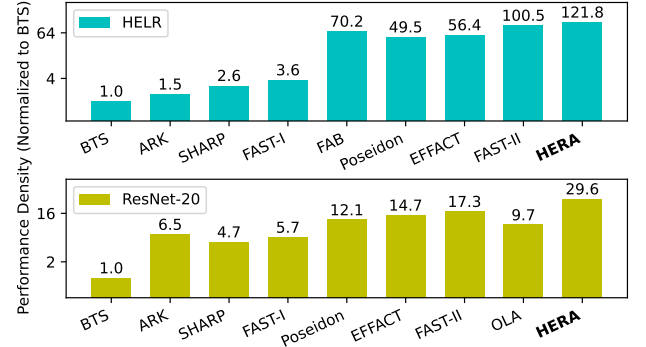
5.2.4 Performance Density. The number of modular multipliers in the design is a key determinant of FHE system throughput. As discussed earlier, HERA employs a small number of 256 multipliers. Fig. 7 presents the performance density of HERA relative to prior accelerators on application benchmarks, normalized to BTS. Performance density is defined as throughput per modular multiplier. For HELR, throughput corresponds to the number of training iterations per second, and for ResNet-20, to the inference frame rate.

In general, FPGA-based accelerators achieve higher performance density due to efficient utilization of compute resources. HERA delivers the highest performance density across both applications. Specifically, it achieves 1.21–2.46× and 1.71–3.05× higher performance density than SOTA FPGA accelerators on HELR and ResNet-20, respectively. Furthermore, compared to ASIC designs, HERA provides an average of 70.9× higher performance density on HELR and 11.4× on ResNet-20. The smaller improvement on ResNet-20 is attributed to the greater complexity and memory intensity of homomorphic convolutions.

6 Related Work

CPU/GPU Acceleration. Prior works have accelerated FHE on CPUs and GPUs. CPU libraries [4, 17, 22, 31, 35] improve performance but remain limited by compute resources. GPUs achieve over 100× speedup against CPU in bootstrapping by exploiting memory bandwidth [21], yet general-purpose platforms still suffer from inflexible scheduling and lack of explicit dataflow control, leading to poor performance on complex FHE applications.

FPGA/ASIC Acceleration. Dedicated accelerators have been explored to address the heavy computation and memory demands of FHE. ASIC-based designs [14, 23, 24, 26, 34] use specialized units

**Figure 7: Performance density comparison on applications**

and large SRAM to achieve high throughput, often two orders of magnitude faster than GPUs, but at the cost of area, power, and design flexibility. FPGA-based accelerators [3, 20, 42, 43, 45] focus on resource efficiency by reusing arithmetic units, yet remain limited by underutilized bandwidth and underexplored memory optimizations. In contrast, HERA exploits FPGA capability through full bandwidth utilization and HMOs that maximize on-chip data reuse within the available SRAM capacity. HEAP [1] uses a combination of CKKS and TFHE schemes, which is orthogonal to this work.

7 Conclusion

In this paper, we presented HERA, an HBM-based FPGA accelerator for FHE with superior bandwidth and memory efficiency. HERA fully exploits the capability of a single FPGA for full-stack FHE acceleration by maximizing on-chip data reuse, minimizing costly off-chip traffic, and saturating available HBM bandwidth. On an Alveo U280, HERA achieves substantial speedups over SOTA FPGA accelerators in bootstrapping and delivers much higher performance density than leading ASICs across end-to-end applications. Overall, our results demonstrate that holistic optimizations and judicious allocation of scarce on-chip resources can unlock the full potential of a single FPGA for practical FHE acceleration.

Acknowledgments

This work was supported by the U.S. National Science Foundation (NSF) under grants CSSI-2311870 and OAC-2505107.

Distribution Statement A: Approved for public release. Distribution is unlimited.

References

- [1] Rashmi Agrawal, Anantha Chandrakasan, and Ajay Joshi. 2024. Heap: A fully homomorphic encryption accelerator with parallelized bootstrapping. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 756–769.
- [2] Rashmi Agrawal, Leo De Castro, Chiraag Juvekar, Anantha Chandrakasan, Vinod Vaikuntanathan, and Ajay Joshi. 2023. Mad: Memory-aware design techniques for accelerating fully homomorphic encryption. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 685–697.
- [3] Rashmi Agrawal, Leo de Castro, Guowei Yang, Chiraag Juvekar, Rabia Yazicigil, Anantha Chandrakasan, Vinod Vaikuntanathan, and Ajay Joshi. 2023. FAB: An FPGA-based accelerator for bootstrappable fully homomorphic encryption. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 882–895.
- [4] Ahmad Al Badawi, Andreea Alexandru, Jack Bates, Flavio Bergamaschi, David Bruce Cousins, Saroja Erabelli, Nicholas Genise, Shai Halevi, Hamish Hunt, Andrey Kim, Yongwoo Lee, Zeyu Liu, Daniele Micciancio, Carlo Pascoe, Yuriy Polyakov, Ian Quah, Saraswathy R.V., Kurt Rohloff, Jonathan Saylor, Dmitriy Suponitsky, Matthew Triplett, Vinod Vaikuntanathan, and Vincent Zucca. 2022. OpenFHE: Open-Source Fully Homomorphic Encryption Library. *Cryptology ePrint Archive*, Paper 2022/915. <https://eprint.iacr.org/2022/915> <https://eprint.iacr.org/2022/915>.
- [5] David H Bailey. 1989. FFTs in external of hierarchical memory. In *Proceedings of the 1989 ACM/IEEE conference on Supercomputing*. 234–242.
- [6] Fabian Boemer, Sejun Kim, Gelila Seifu, Fillipe DM de Souza, and Vinodh Gopal. 2021. Intel HEXL: accelerating homomorphic encryption with Intel AVX512-IFMA52. In *Proceedings of the 9th on Workshop on Encrypted Computing & Applied Homomorphic Cryptography*. 57–62.
- [7] Jean-Philippe Bossuat, Christian Mouchet, Juan Troncoso-Pastoriza, and Jean-Pierre Hubaux. 2021. Efficient bootstrapping for approximate homomorphic encryption with non-sparse keys. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 587–617.
- [8] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. 2018. A full RNS variant of approximate homomorphic encryption. In *International Conference on Selected Areas in Cryptography*. Springer, 347–368.
- [9] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. 2017. Homomorphic encryption for arithmetic of approximate numbers. In *International conference on the theory and application of cryptology and information security*. Springer, 409–437.
- [10] Leo de Castro, Rashmi Agrawal, Rabia Yazicigil, Anantha Chandrakasan, Vinod Vaikuntanathan, Chiraag Juvekar, and Ajay Joshi. 2021. Does fully homomorphic encryption need compute acceleration? *arXiv preprint arXiv:2112.06396* (2021).
- [11] Li Deng. 2012. The mnist database of handwritten digit images for machine learning research [best of the web]. *IEEE signal processing magazine* 29, 6 (2012), 141–142.
- [12] Jiangbin Dong, Xinhua Chen, and Mingyu Gao. 2025. A Unified Vector Processing Unit for Fully Homomorphic Encryption. In *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 1–7.
- [13] Austin Ebel, Karthik Garimella, and Brandon Reagen. 2025. Orion: A Fully Homomorphic Encryption Framework for Deep Learning. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 734–749.
- [14] Shengyu Fan, Xianglong Deng, Liang Kong, Guiming Shi, Guang Fan, Dan Meng, Rui Hou, and Mingzhe Zhang. 2025. FAST: An FHE Accelerator for Scalable-parallelism with Tunable-bit. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA)*. 92–106.
- [15] Shengyu Fan, Zhiwei Wang, Weizhi Xu, Rui Hou, Dan Meng, and Mingzhe Zhang. 2023. Tensorfhe: Achieving practical computation on encrypted data using gpgpu. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 922–934.
- [16] Shai Halevi and Victor Shoup. 2015. Bootstrapping for HELib. In *Advances in Cryptology – EUROCRYPT 2015*, Elisabeth Oswald and Marc Fischlin (Eds.). Springer Berlin Heidelberg, 641–670.
- [17] Shai Halevi and Victor Shoup. 2020. Design and implementation of HELib: a homomorphic encryption library. *Cryptology ePrint Archive* (2020).
- [18] Kyoohyung Han, Seungwan Hong, Jung Hee Cheon, and Daejun Park. 2018. Efficient logistic regression on large encrypted data. *Cryptology ePrint Archive* (2018).
- [19] Kyoohyung Han, Seungwan Hong, Jung Hee Cheon, and Daejun Park. 2019. Logistic regression on homomorphic encrypted data at scale. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 9466–9471.
- [20] Yi Huang, Xincheng Gong, Xiangyu Kong, Dibe Chen, Jianfeng Zhu, Wenping Zhu, Liangwei Li, Mingyu Gao, Shaoyun Wei, Aoyang Zhang, et al. 2025. EF-FACT: A Highly Efficient Full-Stack FHE Acceleration Platform. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1143–1157.
- [21] Wonkyung Jung, Sangpyo Kim, Jung Ho Ahn, Jung Hee Cheon, and Younho Lee. 2021. Over 100x faster bootstrapping in fully homomorphic encryption through memory-centric optimization with GPUs. *IACR Transactions on Cryptographic Hardware and Embedded Systems* (2021), 114–148.
- [22] Wonkyung Jung, Eojin Lee, Sangpyo Kim, Jongmin Kim, Namhoon Kim, Keewoo Lee, Chohong Min, Jung Hee Cheon, and Jung Ho Ahn. 2021. Accelerating fully homomorphic encryption through architecture-centric analysis and optimization. *IEEE Access* 9 (2021), 98772–98789.
- [23] Jongmin Kim, Sangpyo Kim, Jaewan Choi, Jaiyoung Park, Donghwan Kim, and Jung Ho Ahn. 2023. SHARP: A short-word hierarchical accelerator for robust and practical fully homomorphic encryption. In *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*. 1–15.
- [24] Jongmin Kim, Gwangho Lee, Sangpyo Kim, Gina Sohn, Minsoo Rhu, John Kim, and Jung Ho Ahn. 2022. Ark: Fully homomorphic encryption accelerator with runtime data generation and inter-operation key reuse. In *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1237–1254.
- [25] Miran Kim and Kristin Lauter. 2015. Private genome analysis through homomorphic encryption. *BMC medical informatics and decision making* 15, Suppl 5 (2015), S3.
- [26] Sangpyo Kim, Jongmin Kim, Michael Jaemin Kim, Wonkyung Jung, John Kim, Minsoo Rhu, and Jung Ho Ahn. 2022. Bts: An accelerator for bootstrappable fully homomorphic encryption. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA)*. 711–725.
- [27] Sunwoong Kim, Keewoo Lee, Wonhee Cho, Jung Hee Cheon, and Rob A Rutenbar. 2019. FPGA-based accelerators of fully pipelined modular multipliers for homomorphic encryption. In *2019 International Conference on ReConfigurable Computing and FPGAs (ReConFig)*. IEEE, 1–8.
- [28] Alex Krizhevsky, Geoffrey Hinton, et al. 2009. Learning multiple layers of features from tiny images.(2009).
- [29] Dilshan Kumarathunga, Qilin Hu, and Zhenman Fang. 2025. AutoNTT: Automatic Architecture Design and Exploration for Number Theoretic Transform Acceleration on FPGAs. In *2025 IEEE 33rd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 1–9.
- [30] Joon-Woo Lee, HyungChul Kang, Yongwoo Lee, Woosuk Choi, Jieun Eom, Maxim Deryabin, Eunsang Lee, Junghyun Lee, Donghoon Yoo, Young-Sik Kim, et al. 2022. Privacy-preserving machine learning with fully homomorphic encryption for deep neural network. *IEEE Access* 10 (2022), 30039–30054.
- [31] Christian Vincent Mouchet, Jean-Philippe Bossuat, Juan Ramón Troncoso-Pastoriza, and Jean-Pierre Hubaux. 2023. Lattigo v5. Online: <https://github.com/tuneinsight/lattigo>. EPFL-LDS, Tune Insight SA.
- [32] M Sadeq Riaz, Kim Laine, Blake Peltan, and Wei Dai. 2020. HEAX: An architecture for computing on encrypted data. In *Proceedings of the 25th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 1295–1309.
- [33] Nikola Samardzic, Axel Feldmann, Aleksandar Krastev, Srinivas Devadas, Ronald Dreslinski, Christopher Peikert, and Daniel Sanchez. 2021. F1: A fast and programmable accelerator for fully homomorphic encryption. In *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 238–252.
- [34] Nikola Samardzic, Axel Feldmann, Aleksandar Krastev, Nathan Manohar, Nicholas Genise, Srinivas Devadas, Karim Eldefrawy, Chris Peikert, and Daniel Sanchez. 2022. Craterlake: a hardware accelerator for efficient unbounded computation on encrypted data. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA)*. 173–187.
- [35] SEAL 2023. Microsoft SEAL (release 4.1). <https://github.com/Microsoft/SEAL>. Microsoft Research, Redmond, WA..
- [36] Kaustubh Shivdhar, Yuhui Bao, Rashmi Agrawal, Michael Shen, Gilbert Jonatan, Evelio Mora, Alexander Ingare, Neal Livesay, José L. Abellán, John Kim, et al. 2023. Gme: Gpu-based microarchitectural extensions to accelerate homomorphic encryption. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 670–684.
- [37] Ashwajit Singh, Zhihan Xu, and Viktor K Prasanna. 2025. NTT-SAA: Exploring NTT Acceleration with 2-D Systolic Array Architecture on FPGAs. In *2025 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 1–7.
- [38] Olamide Timothy Tawose, Jun Dai, Lei Yang, and Dongfang Zhao. 2023. Toward efficient homomorphic encryption for outsourced databases through parallel caching. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–23.
- [39] Cheng Wang and Mingyu Gao. 2023. Sam: A scalable accelerator for number theoretic transform using multi-dimensional decomposition. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [40] Zhihan Xu, Rajgopal Kannan, and Viktor K Prasanna. 2025. FAME: FPGA Acceleration of Secure Matrix Multiplication with Homomorphic Encryption. *arXiv preprint arXiv:2512.15515* (2025).
- [41] Zhihan Xu, Yang Yang, Rajgopal Kannan, and Viktor K Prasanna. 2024. Bandwidth Efficient Homomorphic Encrypted Discrete Fourier Transform Acceleration on FPGA. In *2024 IEEE 32nd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 1–12.

- [42] Zhihan Xu, Tian Ye, Rajgopal Kannan, and Viktor K Prasanna. 2025. FAST: FPGA Acceleration of Fully Homomorphic Encryption with Efficient Bootstrapping. In *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*. 115–126.
- [43] Yang Yang, Rajgopal Kannan, and Viktor K Prasanna. 2025. OLA: An FPGA-based Overlay Accelerator for Privacy Preserving Machine Learning with Homomorphic Encryption. In *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*. 127–138.
- [44] Yang Yang, Weihang Long, Rajgopal Kannan, and Viktor K Prasanna. 2023. Fpga acceleration of rotation in homomorphic encryption using dynamic data layout. In *2023 33rd International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 174–181.
- [45] Yinghao Yang, Huaizhi Zhang, Shengyu Fan, Hang Lu, Mingzhe Zhang, and Xiaowei Li. 2023. Poseidon: Practical homomorphic encryption accelerator. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 870–881.
- [46] Songnian Zhang, Suprio Ray, Rongxing Lu, Yunguo Guan, Yandong Zheng, and Jun Shao. 2022. Efficient and privacy-preserving spatial keyword similarity query over encrypted data. *IEEE Transactions on Dependable and Secure Computing* 20, 5 (2022), 3770–3786.
- [47] Junwei Zhou, Botian Lei, Huile Lang, Emmanouil Panaousis, Kaitai Liang, and Jianwen Xiang. 2023. Secure genotype imputation using homomorphic encryption. *Journal of information security and applications* 72 (2023), 103386.